

Homework 5: Vision Transformers for Image Classification

Gilberto Feliu
Student ID: 801257813

July 20, 2026

Assignment: Homework 5

GitHub Repository: <https://github.com/gfeliu-rgb/ECGR-5106-HW5>

Late Submission Note

This homework was submitted late because I had a family medical emergency. My grandmother has pancreatic cancer and was hospitalized during the week of the deadline, which disrupted the time I had planned for completing and packaging the assignment.

Dataset and Experimental Setup

Both problems use CIFAR-100 through `torchvision.datasets.CIFAR100`. Training images are normalized with CIFAR-100 channel statistics. Problem 1 uses 32 by 32 inputs with random crop, padding, and horizontal flip for training; evaluation uses only tensor conversion and normalization. Problem 2 uses 224 by 224 inputs for the pretrained Swin rows. The scratch Swin run was completed as a CPU fallback with native 32 by 32 CIFAR-100 inputs so that a full five-epoch scratch baseline could be measured in the current environment.

The scripts use CPU fallback with `torch.device("cuda" if torch.cuda.is_available() else "cpu")`. The completed CSV summaries record CUDA for all Problem 1 runs and for the two completed pretrained Swin runs. The current verification session did not have a usable CUDA device, so the completed training runs were not repeated here. Package versions used in the current environment were: `torch 2.12.0+cu130`, `torchvision 0.27.0+cu130`, `transformers 5.14.1`, `pandas 2.2.3`, `numpy 2.2.6`, and `matplotlib 3.10.9`.

Important status note: Problem 1 has 10-epoch results for all planned rows. Problem 2 now has completed scratch Swin metrics, but Swin-Tiny and Swin-Small still reflect one finished epoch rather than the originally planned five-epoch schedule.

Reproducibility and Metric Details

CIFAR-100 contains 50,000 training images and 10,000 test images across 100 classes. The scripts train on the official training split and evaluate on the official test split. The CSV column names use `val_loss` and `val_accuracy_pct`, but these values are computed on the CIFAR-100 test split because no separate validation split was created for this homework. Randomness is controlled with seed 4106 for Python and PyTorch.

All models use cross-entropy loss and top-1 accuracy. Training time is measured as average wall-clock seconds per epoch, including evaluation at the end of each epoch. The history CSV files in the

two results folders contain the per-epoch losses, accuracies, and runtime values used for the plots. The main commands used to reproduce the artifacts are:

```
python src/hw5_vit_resnet.py --run-all-vit --run-resnet
python src/hw5_swin.py --run-pretrained swin_tiny_pretrained
python src/hw5_swin.py --run-pretrained swin_small_pretrained
python src/hw5_swin.py --run-scratch --epochs 5 --batch-size 128 \
    --image-size 32 --num-threads 4
python tools/make_hw5_plots.py
```

Problem 1: Vision Transformer from Scratch vs. ResNet-18

Objective

The goal of Problem 1 is to compare several Vision Transformer configurations against a ResNet-18 baseline on CIFAR-100 and discuss accuracy, parameter count, estimated FLOPs, and training time.

Implementation Summary

The custom ViT starts with a convolutional patch embedding layer whose kernel size and stride match the patch size. The flattened patch sequence is prepended with a learned class token, then combined with a learned positional embedding. The encoder is built from `nn.TransformerEncoderLayer` using GELU activations, multi-head self-attention, feed-forward MLP blocks, dropout, and layer normalization. The classifier uses the final normalized class-token representation and a linear head for 100 CIFAR-100 classes.

The ResNet-18 baseline uses `torchvision.models.resnet18(weights=None)`. Its first convolution is changed to a 3 by 3 stride-1 convolution and the initial max-pooling layer is removed so the model is better matched to 32 by 32 CIFAR images. All models use cross-entropy loss and Adam optimization. Evaluation runs without gradients on the CIFAR-100 test split and records validation/test loss and top-1 accuracy.

Results

Model	Params	FLOPs	Time/Epoch	Val Loss	Test Acc.	Status
ViT-A	3,214,692	2.13e8	23.01	3.2834	20.01%	10 epochs, CUDA
ViT-B	25,330,276	1.67e9	106.72	4.3232	4.51%	10 epochs, CUDA
ViT-C	3,239,268	5.41e7	12.22	3.8851	9.68%	10 epochs, CUDA
ViT-D	25,379,428	4.30e8	42.30	4.2694	4.95%	10 epochs, CUDA
ResNet-18	11,220,132	1.10e9	61.24	1.3919	61.20%	10 epochs, CUDA

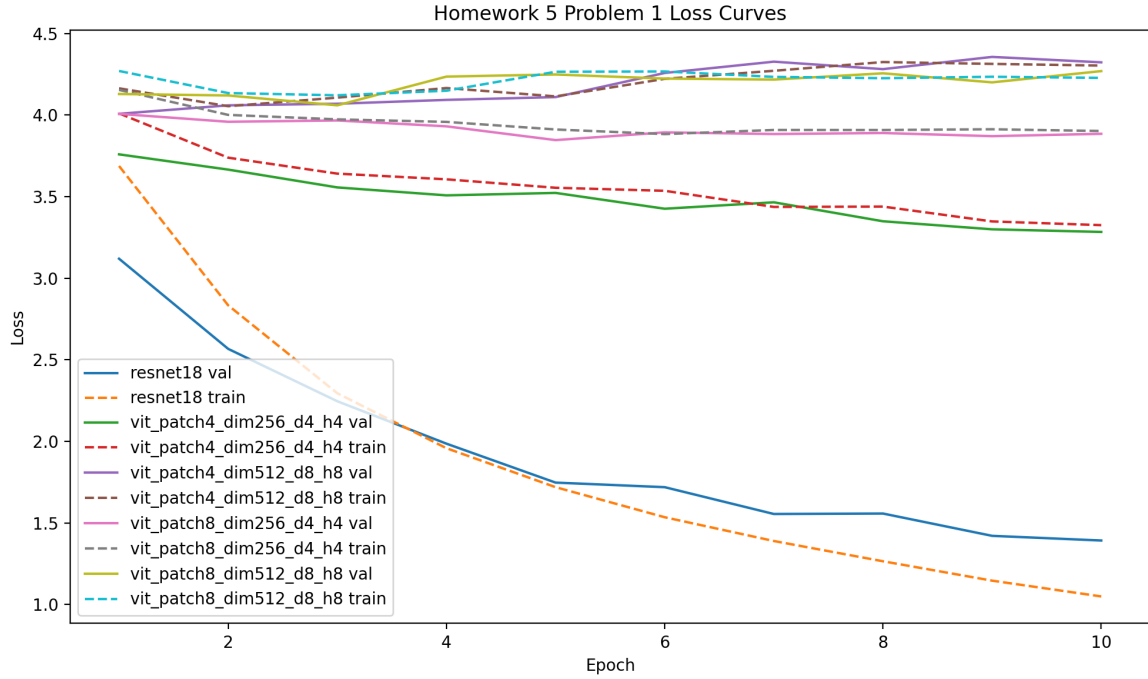
Analysis

Among the ViT models, ViT-A performed best with 20.01% test accuracy and a final validation loss of 3.2834. ViT-C was the next best ViT at 9.68% while using the lowest estimated FLOP count. ViT-B and ViT-D were much larger, about 25.3 million trainable parameters each, but performed poorly in this 10-epoch run. The larger transformer settings appear harder to optimize from scratch on CIFAR-100 under this short schedule.

Patch size affected compute directly. With 32 by 32 inputs, patch size 4 creates 64 image tokens plus the class token, while patch size 8 creates 16 image tokens plus the class token. Since transformer attention scales strongly with sequence length, the patch size 8 configurations had much lower estimated FLOPs and lower runtime. The tradeoff is that the lower-token models also had weaker accuracy in this run.

The ResNet-18 baseline was strongest overall, reaching 61.20% test accuracy after 10 epochs. This result is expected on CIFAR-100 because convolutional inductive bias helps ResNet learn local image

features efficiently, while a from-scratch ViT generally needs more data, longer schedules, stronger regularization, or pretraining to become competitive.



Problem 2: Fine-Tuning Pretrained Swin Transformers vs. Training from Scratch

Objective

The goal of Problem 2 is to compare pretrained Swin-Tiny and Swin-Small head-only fine-tuning against a scratch Swin-style model on CIFAR-100.

Implementation Summary

The pretrained Swin models are loaded from Hugging Face using the `SwinForImageClassification` loader. The classifier head is replaced for 100 CIFAR-100 classes with `ignore_mismatched_sizes=True`. For the pretrained experiments, all backbone parameters are frozen and only classifier parameters remain trainable. The script uses Adam, cross-entropy loss, batch size 32, learning rate $2e-5$, and 224 by 224 resized CIFAR-100 images.

The scratch model uses `torchvision.models.swin_t(weights=None)` with its head replaced for 100 classes. Unlike the pretrained runs, all scratch-model parameters are trainable. Because CUDA was not available in the verification session, the measured scratch run uses native 32 by 32 CIFAR-100 inputs, batch size 128, learning rate 0.001, and 5 epochs on CPU.

Results

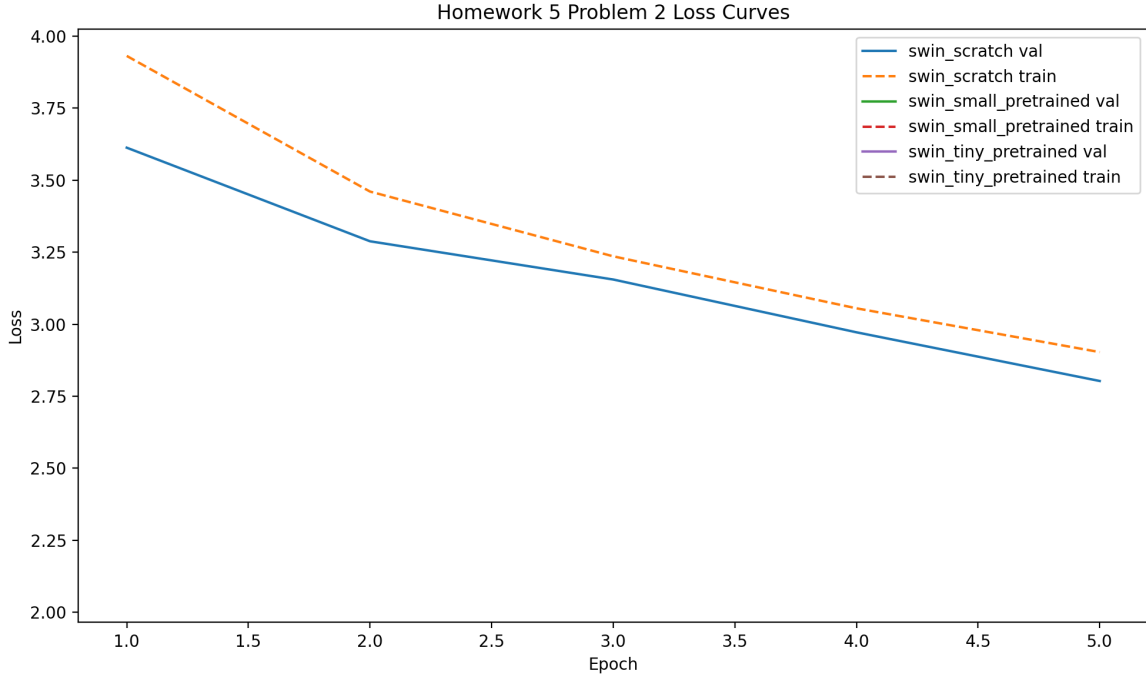
Model	Pretrained	Frozen	Total Params	Trainable Params	Time/Epoch	Val Loss	Test Acc.	Status
Swin-Tiny	Yes	Yes	27,596,254	76,900	851.96	2.2969	60.56%	1 epoch, CUDA
Swin-Small	Yes	Yes	48,914,158	76,900	678.62	2.0581	64.74%	1 epoch, CUDA
Scratch Swin	No	No	27,596,254	27,596,254	1244.41	2.8027	29.19%	5 epochs, CPU, 32x32

Analysis

The completed pretrained runs show a clear transfer-learning advantage after one epoch. Swin-Small reached 64.74% test accuracy, and Swin-Tiny reached 60.56%. These accuracies are much higher than the from-scratch ViT and ResNet one-epoch results in Problem 1 because the Swin backbones already contain useful image features from pretraining.

Swin-Small performed better than Swin-Tiny in the completed artifacts, with lower validation loss and higher accuracy. The trainable parameter count is the same for both pretrained rows because the backbone is frozen and only the classifier head is trained. The total parameter count is higher for Swin-Small, so it has a larger memory footprint even though the optimizer updates the same 76,900 classifier parameters.

The scratch Swin baseline reached 29.19% accuracy after five CPU epochs. This is well below both pretrained Swin rows even though the pretrained rows only ran for one epoch, which supports the expected advantage of transfer learning. The comparison is not perfectly controlled because the scratch row used native 32 by 32 inputs and CPU timing, while the pretrained rows used 224 by 224 inputs and CUDA. The accuracy gap is still large enough to show that the pretrained representations are much more effective under a short schedule.



Conclusion

The results support two main takeaways. First, ResNet-18 substantially outperformed all from-scratch ViT configurations after 10 epochs on CIFAR-100, and ViT-A was the best transformer configuration among the four tested. Second, pretrained Swin feature extractors were much stronger than scratch Swin even with fewer completed epochs, showing the practical value of transfer learning under a short training schedule.

To exactly match the intended Problem 2 schedule, the remaining work is to rerun Swin-Tiny and Swin-Small for the full five epochs on a machine with a usable GPU.